Logical Planning Laboratory Report

Course: CS F407 Artificial Intelligence. **Files:** `logic_lab.py` (planner), `logic_tests.py` (tests), `planner.pl` (Prolog, run with SWI-Prolog 9.0.4). All results were produced by running these files.

Task 0. The planning problem (I, A, G)

(a) Initial state: I = {At(Robot,A), At(Package,A)}. **(b) Goal:** G = {At(Package,C)}. **(c) Actions:** Move between connected locations (A-B, B-A, B-C, C-B); PickUp(Package,l) and Drop(Package,l) for l in {A,B,C}.

Action	Preconditions	Effects
Move(x,y)	At(Robot,x)	not At(Robot,x); At(Robot,y)
PickUp(Package,l)	At(Robot,l), At(Package,l)	not At(Package,l); Holding(Package)
Drop(Package,l)	At(Robot,l), Holding(Package)	not Holding(Package); At(Package,l)

Applicable in I (computed by the planner): Move(A,B), PickUp(Package,A). **PickUp(Package,A) is applicable**: its preconditions At(Robot,A) and At(Package,A) are both in I, so I satisfies them. **Drop(Package,C) is not applicable**: it needs At(Robot,C) and Holding(Package), and neither is true in I. Appearing in the action list is not enough; every precondition must hold in the current state.

Task 1. Plan constructed by hand

State	After action	Facts
S0	-	At(Package,A), At(Robot,A)
S1	PickUp(Package,A)	At(Robot,A), Holding(Package)
S2	Move(A,B)	At(Robot,B), Holding(Package)
S3	Move(B,C)	At(Robot,C), Holding(Package)
S4	Drop(Package,C)	At(Package,C), At(Robot,C)

Plan: PickUp(Package,A), Move(A,B), Move(B,C), Drop(Package,C); the last state satisfies G. **Note on the lab's example:** the lab suggests thinking about Move(A,B) followed by PickUp(Package,B), but the package is at A, so PickUp(Package,B) is *not* applicable (checked: False). The package must be picked up at A first, then carried.

Task 2. Prompt given to the LLM

[Personalise: replace with the exact prompt you used and say which parts you accepted or changed]

```
I want to implement a simple planning agent in Python. Represent a state as a set of
logical propositions. Each action has a name, positive preconditions, negative
preconditions, positive effects and negative effects. An action is applicable if all its
preconditions are satisfied by the current state. Applying an action removes its negative
effects and adds its positive effects. Use breadth-first search to find a sequence of
actions achieving a goal. Detect when no plan exists, print the action sequence and the
state after each action. Explain the implementation and state your assumptions.
Then run it on the warehouse problem (locations A,B,C; Move/PickUp/Drop as specified).
```

Think About It: where the ideas appear in logic_lab.py. Preconditions: `applicable()`. Effects: `apply()`. Goal: `goal <= s` in `bfs_plan()`. BFS: the FIFO `deque` frontier and the `parent` map used to rebuild the plan.

Task 3. Tests

Test	Initial state	Goal	Plan found?	Plan	Valid?
A Solvable	At(Robot,A), At(Package,A)	At(Package,C)	Yes	PickUp(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C)	Yes, independent_check passed all 4 steps
B Impossible (PickUp removed)	same	At(Package,C)	No: 'No plan found'	-	n/a (correct: goal is unreachable)

Test	Initial state	Goal	Plan found?	Plan	Valid?
C1 Robot can reach C, package cannot move	same, no PickUp	At(Package,C)	No: 'No plan found'	-	correct: robot at C is not package at C
C2 Same actions, robot goal	same, no PickUp	At(Robot,C)	Yes	Move(A,B) -> Move(B,C)	Yes
C3 Decoy action Wander(A,B) (moves robot only)	same, full actions	At(Package,C)	Yes	PickUp(Package,A) -> Move(A,B) -> Move(B,C) -> Drop(Package,C)	Yes; the decoy is not used

Test A: independent verification (a second function with its own bookkeeping) shows each action's preconditions held in the state where it was executed:

```
PickUp(Package,A)    -> At(Robot,A), Holding(Package)
Move(A,B)            -> At(Robot,B), Holding(Package)
Move(B,C)            -> At(Robot,C), Holding(Package)
Drop(Package,C)      -> At(Package,C), At(Robot,C)
```

Tests C1/C2 show the planner does not confuse the robot reaching C with the package reaching C: the goal test checks the stated goal fact only.

Task 4. Logic and search

The missing step in the diagram is: **if S |= Pre(a), apply the effects (remove the negative effects, add the positive effects)**, giving S' = Apply(S,a). Logical reasoning decides *whether* an action is applicable and *what the new state is*; search decides *which* applicable action sequences to try and in what order, keeping a frontier of states and remembering visited ones. Together, search expands only logically possible successors, and the first state satisfying the goal gives the plan. *Logic determines what is possible; search determines what to try.*

Task 5. Can the LLM verify its own plan?

LLM-style explanation: 'PickUp(Package,A): needs At(Robot,A) and At(Package,A); both are in S0. Move(A,B): needs At(Robot,A); holds in S1. Move(B,C): needs At(Robot,B); holds in S2. Drop(Package,C): needs At(Robot,C) and Holding(Package); both hold in S3; result At(Package,C).' **Independent execution** (table above) gives exactly the same states, so here the two agree.

[Personalise: paste your own LLM's explanation and compare it with the executed transitions]

Which to trust more: (b), the independently executed transitions. An explanation is generated text that reads plausibly whether or not it is true; the executed check actually evaluates the preconditions against the real states and would reveal a wrong step. Agreement increases confidence, but only the executed check is verification.

Reflection questions

1. Writing preconditions and effects first fixes what 'correct' means, so the generated planner can be tested against it; otherwise the LLM would invent its own rules. **2.** Without precondition checks the planner could, for example, Drop the package at C while the robot is still at A, or PickUp a package from another location, producing 'plans' that cannot be executed. **3.** A plan can look sensible but contain an inapplicable step (e.g. PickUp(Package,B) after Move(A,B)); validity is only established by checking each step. **4.** The LLM contributed boilerplate: data structures, the BFS loop and printing. **5.** I verified independently: the semantics of apply, the plan's step-by-step validity, the no-plan cases, and the robot/package distinction. **6.** In checking S |= Preconditions(a) (applicable()), in the goal test, and in the Prolog verifier. **7.** Planning is search through the space of states: states are nodes, applicable actions are edges, and BFS finds a shortest plan just like the grid-search lab.

[Personalise: questions 4 and 5 should describe your own session]

Optional: Prolog as a logical verifier (planner.pl)

```
?- can_move(a,b).   true
?- can_move(a,c).   false
```

```
?- valid_move(a,b).   true
?- valid_move(b,c).   true
?- valid_move(a,c).   false
?- valid_route([a,b,c]).   true
?- valid_route([a,c]).   false
?- reduce_speed.   true
```

Task 6. (a) can_move(a,b) is true because the fact connected(a,b) exists, so the rule body is satisfied. (b) can_move(a,c) is not established because there is no fact connected(a,c) and no rule that derives it; Prolog's 'false' means 'cannot be proved from this knowledge base' (negation as failure), not that it is impossible in all worlds. (c) The rule `can_move(X,Y) :- connected(X,Y).` is the implication Connected(x,y) -> CanMove(x,y), universally quantified, with the head as the conclusion.

Task 7. valid_move(a,b) and valid_move(b,c) succeed and valid_move(a,c) fails, as predicted. **Challenge:** for the proposed Move(a,c), valid_move(a,c) is false, so the knowledge base does not support that action and a plan containing it would be rejected. I also added `valid_route/1` to check a whole route: valid_route([a,b,c]) is true and valid_route([a,c]) is false. Generation (Python planner) and checking (Prolog) are separate, so a mistake in one is unlikely to be repeated in the other.

Task 8. reduce_speed succeeds because wet_road is a fact, slippery follows from it by the first rule, and reduce_speed follows from slippery by the second rule: **wet_road (fact) => slippery (rule) => reduce_speed (rule) => conclusion: reduce_speed.** Prolog works backwards from the query, proving reduce_speed by proving slippery, which needs wet_road.

Prolog reflection. (1) A fact is an unconditional statement (wet_road.); a rule has a head that holds if its body holds (slippery :- wet_road.). (2) A query asks whether the statement follows from the facts and rules. (3) It gives an independent check of whether each proposed move is supported by the stated warehouse knowledge. (4) The verifier does not share the generator's possible mistakes or fabrications, so it can catch an invalid action that the LLM-assisted planner produced.